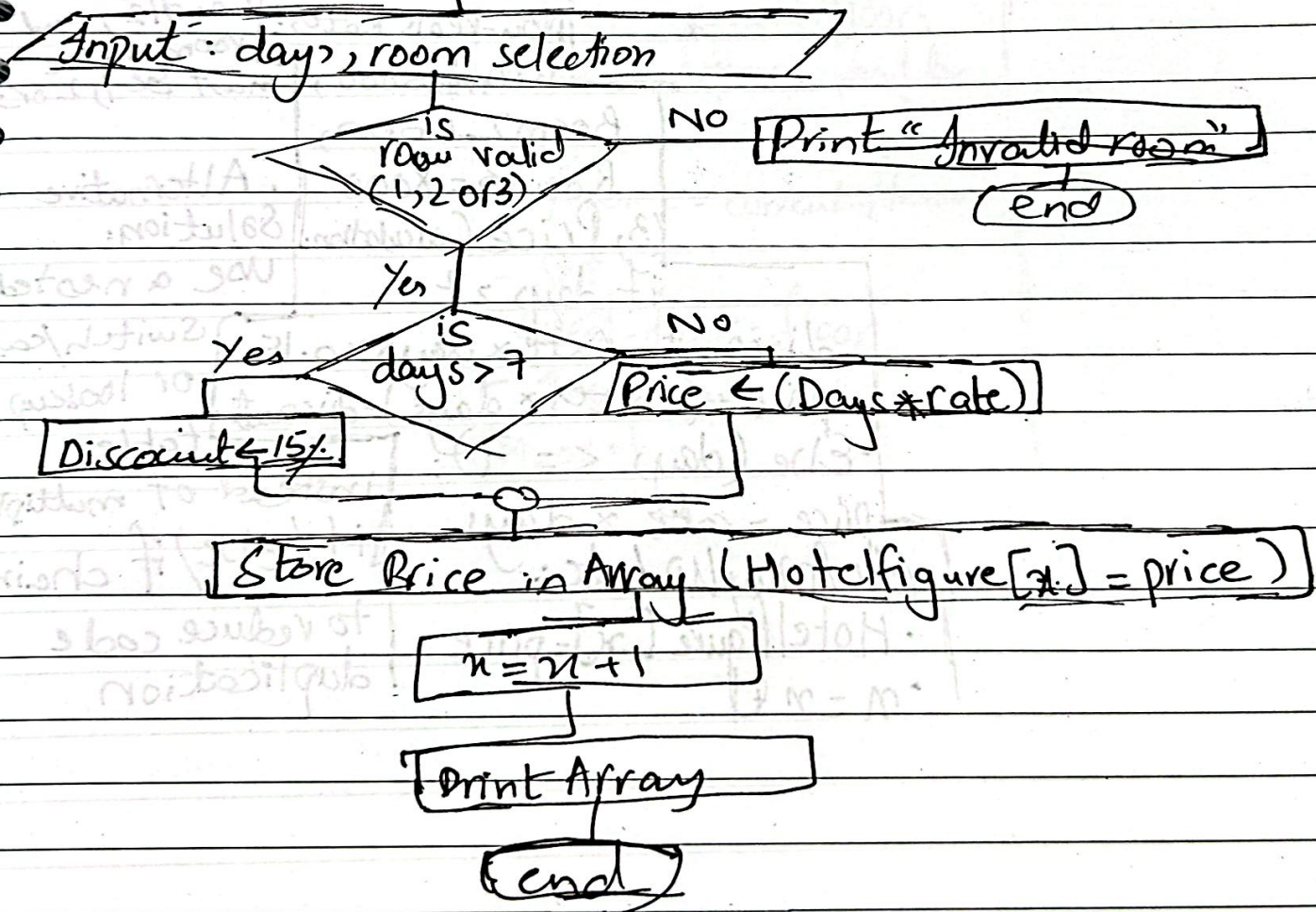
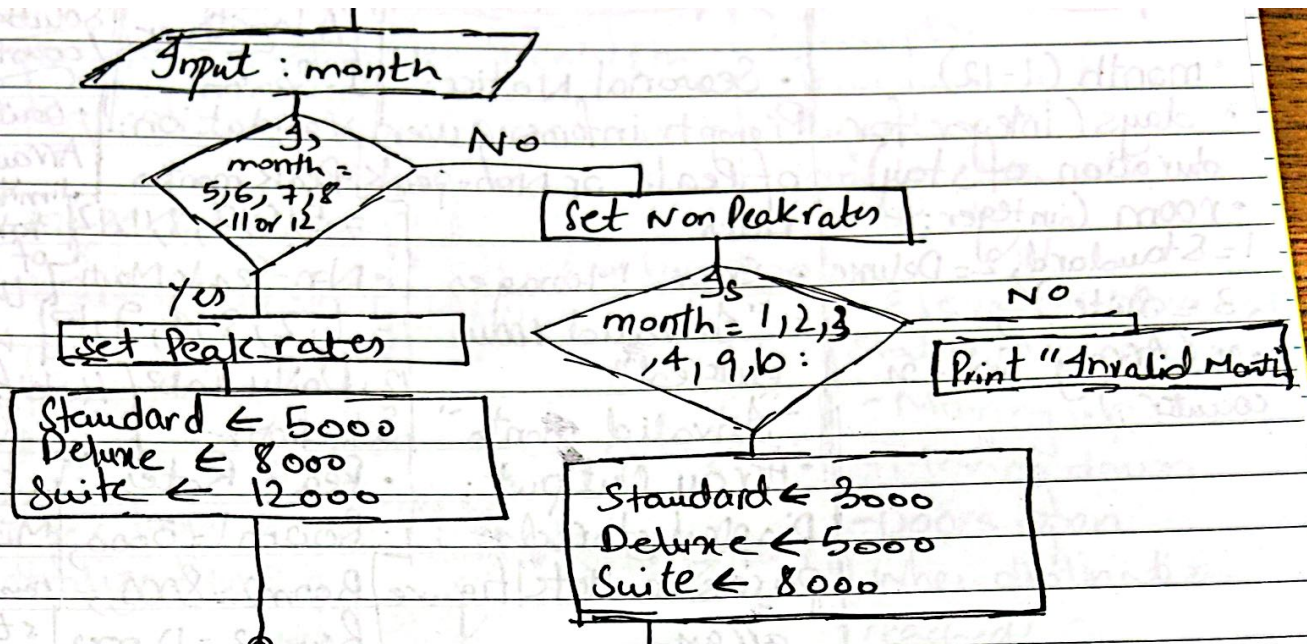


Part B Question 1

4Po:-

Input	Processing	Output
- month (1-12) - days (Number of days) - room (1 = Standard, 2 = Deluxe, 3 = Suite) - n (Array index counter)	1. Season Validation & Pricing assignment: - Peak months (5, 6, 7, 8, 11, 12) - Standard = 5k/day - Deluxe = 8k/day - Suite = 12k/day - No Peak Month (1, 2, 3, 4, 9, 10): - Standard (3k/day) - Deluxe (5k/day) - Suite (8k/day)	User Prompts :- - Seasonal notifications - Room selection menu. - Error messages - "Invalid room entered" - "Invalid month" Final Output:- - Print Hotel figure array contents
	2. Discount and Total Calculation:- - if days > 7: - discount = (rate × days) × 0.15 - price = (rate × days) - discount - if days ≤ 7: - price = rate × days	
	(3) Array storage Hotel figure [n] ← price n = n + 1	



Pac:-

Question 1 Part B :-

Inputs:-

- month (1-12)
- days (integer for duration of stay)
- room (integer: 1 = Standard, 2 = Deluxe, 3 = Suite)
- n (Array index counter)

Required Outputs:-

- Seasonal Notice: Prompts informing user of Peak or Non-peak rates
- Error Messages
 - "Invalid room entered"
 - "Invalid month"
- Array Output: Display stored prices in Hotel figure array.

Processing / Algorithm

1. Season Validation:

Peak months = 5, 6, 7, 8, 11, 12

• Non-Peak Months = 1, 2, 3, 4, 9, 10

2. Daily rate Selection:-

• Peak Rates

Room 1 = 5000,

Room 2 = 8000,

Room 3 = 12000,

Non-Peak Rates:

Room 1 = 3000,

Room 2 = 5000,

Room 3 = 8000

3. Price Calculation:

if days > 7

discount = rate × days × 0.15

price = (rates × days) - discount

• Else (days ≤ 7):

price = rate × days

(4) Array Update:

• Hotel figure [x] = price

• n = n + 1

Alternative Solutions / constraint:-

• Constraint 1:-

Array bounds limit the total number of bookings that can be stored in Hotel figure.

• Constraint 2:-

Month input must be strictly b/w 1 and 12, and room choice must be 1, 2 or 3

Alternative Solution:

Use a nested

Switch/case or lookup table

instead of multiple if/else/if chains to reduce code duplication

Question 2:- APO :-

Input:-

• N (Integer: total number of requests)

• floor (Integer: Destination floor requested by user)

• current_floor (Integer: initialised to 0)

• n (Integer loop counter initialized to 0)

Processing:-

(1) Loop Execution:-

• Repeat process while $n \leq N$

(2) Elevator Direction Logic:-

• if floor > current_floor:

- Increment
current_floor = current_floor + 1

• Else if

- floor < current_floor:

- Decrement
current_floor = current_floor - 1

• Else

floor == current_floor

- Set

current_floor = floor

- Increment

$n = n + 1$

Output:-

• Prompt:

- "How many floor requests"

- "which floor do you want to go"

• Elevator Movement Status Messages:-

- Moving up

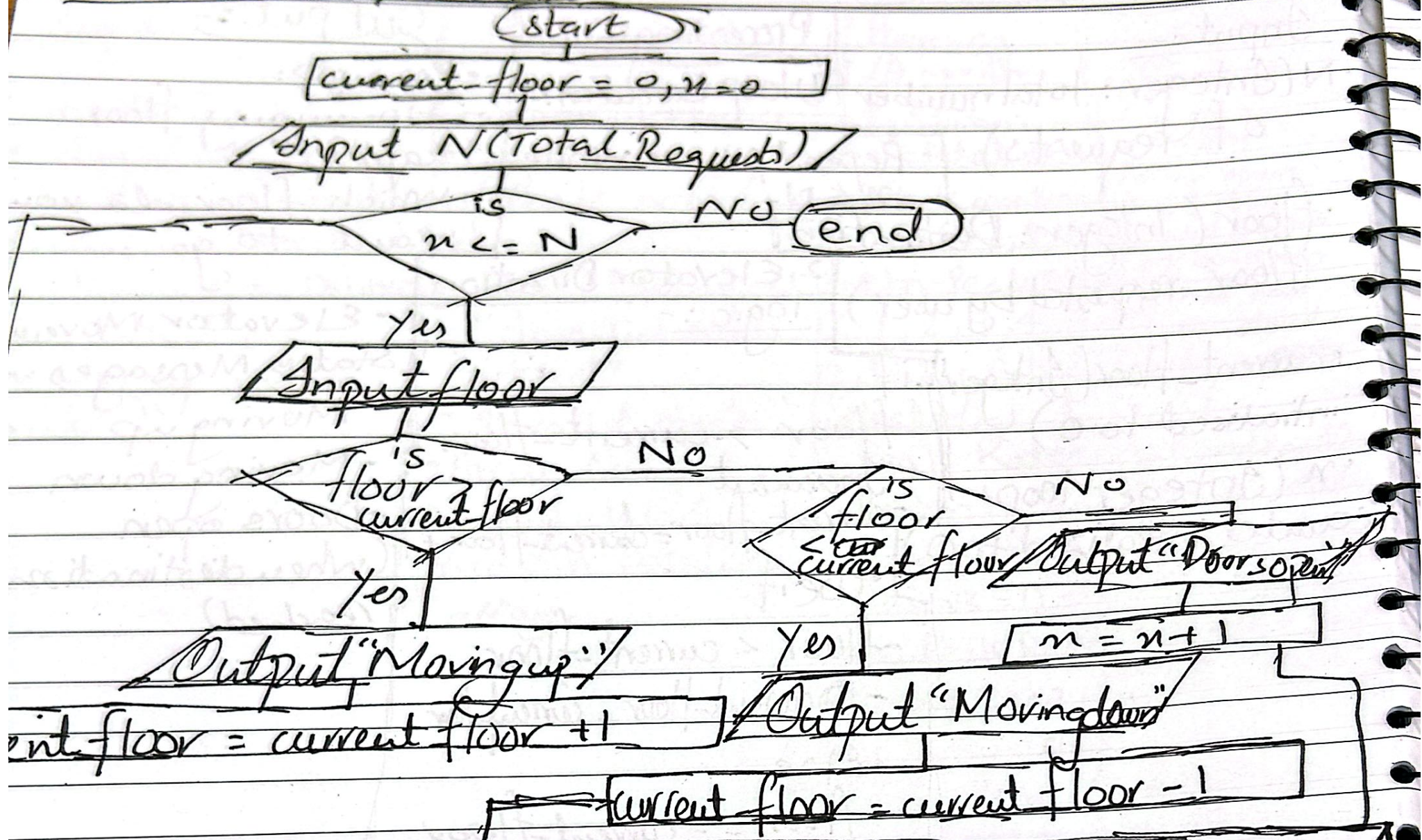
- Moving down

- Doors open

(when destination reached)

Part B:-

Question 2:- flowchart:-



Part B:-
 Pac:-

Question 2:-

Inputs:-	Required Outputs:-	Processing/Algorithms	Alternative Solutions/Constraints:-
• N (total number of floor requests) • floor (target destination floor for each request) • current floor (initialized to 0) • n (Request loop counter initialized to 0)	• Elevator Movement Status Messages: - Upward movement indicator ("Moving up") - Downward movement indicator ("Moving down") • Arrival status: - Door status notification ("Doors open")	1) Request loop: • Execute loop while request counter $n \leq N$ 2) Direction Evaluation & Position Updating: • if $floor > current_floor$: - Print upward motion indicator. - Increment $current_floor = current_floor + 1$ • Else if $floor < current_floor$: - Print downward motion indicator. - Decrement $current_floor = current_floor - 1$ • Else ($floor == current_floor$): - Display arrival notification ("Doors open"). - Update state: $current_floor = floor$ - Increment completed request counter: $n = n + 1$	• Constraint 1: The counter is only incremented when a requested floor is fully reached, ensuring every single request is completely serviced before advancing. • Constraint 2: The algorithm moves one floor step at a time per iteration rather than jumping directly to the target floor. • Alternative solution: Use direct arithmetic $current_floor = floor$ inside a single step instead of stepping floor-by-floor with unit increments/decrements.